

Binary File Handling in Java

29 June 2026 20:30

A binary file stores data in binary (zeros and ones) Rather than as human readable text. Unlike text files, binary files preserve the exact internal representation of data. Making them faster, more compact, and suitable for storing objects, images, audio, video and primitive data types.

For example:

```
Text File(student.txt)
 101 Amit 92.5
 102 Priya 88.0
```

The contents are readable

```
Binary File(student.dat)
```

```
010000001 0110101101 0111101110...
```

The content is not human readable. Sometime it displays symbols also, when we render it on different text editors.

Advantages of Binary Files

- Faster read or write operation.
- Occupy less storage.
- Preserve the exact data type.
- No conversion between text and numeric values.
- Suitable for storing Java objects.

Java Classes Used for Binary File Handling

Java provides several classes for binary file operations.

Class	Purpose
FileInputStream	Reads binary data byte by byte.
FileOutputStream	Writes binary data byte by byte.
BufferedInputStream	Faster buffered reading.
BufferedOutputStream	Faster buffered writing.
DataInputStream	Reads primitive data types.
DataOutputStream	Writes primitive data types.
ObjectInputStream	Reads serialized objects.
ObjectOutputStream	Writes serialized objects.
RandomAccessFile	Reads or writes at any location.

FileOutputStream: We used to write raw bytes into a binary file.

Class FileOutputStream

java.lang.Object
java.io.OutputStream
java.io.FileOutputStream

All Implemented Interfaces:

Closeable, Flushable, AutoCloseable

```
public class FileOutputStream  
extends OutputStream
```

A file output stream is an output stream for writing data to a `File` or to a `FileDescriptor`. Whether or not a file is available or may be created depends upon the underlying platform. Some platforms, in particular, allow a file to be opened for writing by only one `FileOutputStream` (or other file-writing object) at a time. In such situations the constructors in this class will fail if the file involved is already open.

`FileOutputStream` is meant for writing streams of raw bytes such as image data. For writing streams of characters, consider using `FileWriter`.

API Note:

The `close()` method should be called to release resources used by this stream, either directly, or with the try-with-resources statement.

Implementation Requirements:

Subclasses are responsible for the cleanup of resources acquired by the subclass. Subclasses requiring that resource cleanup take place after a stream becomes unreachable should use `Cleaner` or some other mechanism.

Since:

1.0

Constructor Summary

Constructors

Constructor	Description
<code>FileOutputStream(File file)</code>	Creates a file output stream to write to the file represented by the specified <code>File</code> object.
<code>FileOutputStream(FileDescriptor fdObj)</code>	Creates a file output stream to write to the specified file descriptor, which represents an existing connection to an actual file in the file system.
<code>FileOutputStream(File file, boolean append)</code>	Creates a file output stream to write to the file represented by the specified <code>File</code> object.
<code>FileOutputStream(String name)</code>	Creates a file output stream to write to the file with the specified name.
<code>FileOutputStream(String name, boolean append)</code>	Creates a file output stream to write to the file with the specified name.

Method Summary

All Methods

Instance Methods

Concrete Methods

Modifier and Type	Method	Description
void	<code>close()</code>	Closes this file output stream and releases any system resources associated with this stream.
<code>FileChannel</code>	<code>getChannel()</code>	Returns the unique <code>FileChannel</code> object associated with this file output stream.
final <code>FileDescriptor</code>	<code>getFD()</code>	Returns the file descriptor associated with this stream.
void	<code>write(byte[] b)</code>	Writes <code>b.length</code> bytes from the specified byte array to this file output stream.
void	<code>write(byte[] b, int off, int len)</code>	Writes <code>len</code> bytes from the specified byte array starting at offset <code>off</code> to this file output stream.
void	<code>write(int b)</code>	Writes the specified byte to this file output stream.

```
package FileHandling.BinaryFileHandling;  
import java.util.*;  
import java.io.*;  
  
/**  
 * Write a description of class WriteBinary here.  
 *  
 * @author (your name)  
 * @version (a version number or a date)  
 */  
public class WriteBinary  
{  
    public static void main(String[] args){  
        Scanner sc = new Scanner(System.in);  
        System.out.print("\nEnter file name: ");
```

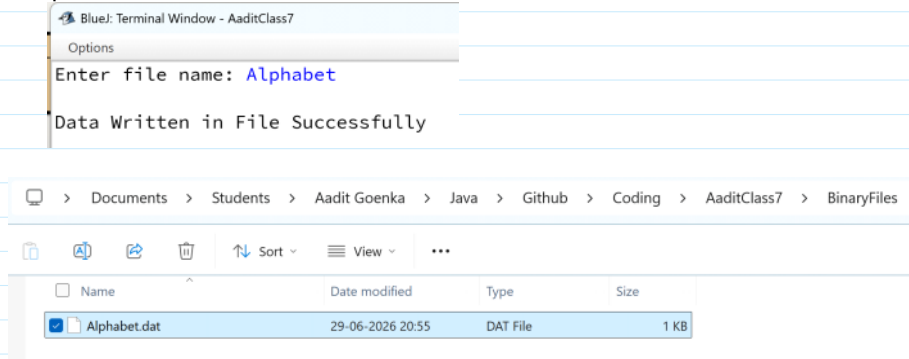
```

String path = "./BinaryFiles/";
String fileName = sc.nextLine();
try(FileOutputStream fos = new
FileOutputStream(path+fileName+".dat", true)){
    fos.write(65);
    fos.write(66);
    fos.write(67);
    fos.write(68);

    fos.close();
    System.out.print("\nData Written in File Successfully");
}
catch(FileNotFoundException fnfe){
    System.err.print("\nFile is not present on the specified
directory.");
}
catch(IOException ioe){
    System.err.print("\nInput/Output network jam.");
}
}
}

```

Output:



FileInputStream: Reads bytes from the binary file.

Class FileInputStream

java.lang.Object
 java.io.InputStream
 java.io.FileInputStream

All Implemented Interfaces:

Closeable, AutoCloseable

public class **FileInputStream**
 extends [InputStream](#)

A **FileInputStream** obtains input bytes from a file in a file system. What files are available depends on the host environment.

FileInputStream is meant for reading streams of raw bytes such as image data. For reading streams of characters, consider using [FileReader](#).

API Note:

The `close()` method should be called to release resources used by this stream, either directly, or with the `try-with-resources` statement.

Implementation Requirements:

Subclasses are responsible for the cleanup of resources acquired by the subclass. Subclasses requiring that resource cleanup take place after a stream becomes unreachable should use [Cleaner](#) or some other mechanism.

Since:

1.0

Constructor Summary

Constructors

Constructor	Description
<code>FileInputStream(File file)</code>	Creates a <code>FileInputStream</code> to read from an existing file represented by the <code>File</code> object <code>file</code> .
<code>FileInputStream(FileDescriptor fdObj)</code>	Creates a <code>FileInputStream</code> by using the file descriptor <code>fdObj</code> , which represents an existing connection to an actual file in the file system.
<code>FileInputStream(String name)</code>	Creates a <code>FileInputStream</code> to read from an existing file named by the path name <code>name</code> .

Method Summary

All Methods

Instance Methods

Concrete Methods

Modifier and Type	Method	Description
<code>int</code>	<code>available()</code>	Returns an estimate of the number of remaining bytes that can be read (or skipped over) from this input stream without blocking by the next invocation of a method for this input stream.
<code>void</code>	<code>close()</code>	Closes this file input stream and releases any system resources associated with the stream.
<code>FileChannel</code>	<code>getChannel()</code>	Returns the unique <code>FileChannel</code> object associated with this file input stream.
<code>final FileDescriptor</code>	<code>getFD()</code>	Returns the <code>FileDescriptor</code> object that represents the connection to the actual file in the file system being used by this <code>FileInputStream</code> .
<code>int</code>	<code>read()</code>	Reads a byte of data from this input stream.
<code>int</code>	<code>read(byte[] b)</code>	Reads up to <code>b.length</code> bytes of data from this input stream into an array of bytes.
<code>int</code>	<code>read(byte[] b, int off, int len)</code>	Reads up to <code>len</code> bytes of data from this input stream into an array of bytes.
<code>byte[]</code>	<code>readNBytes(int len)</code>	Reads up to a specified number of bytes from the input stream.
<code>long</code>	<code>skip(long n)</code>	Skips over and discards <code>n</code> bytes of data from the input stream.

read

```
public int read()
    throws IOException
```

Reads a byte of data from this input stream. This method blocks if no input is yet available.

Specified by:

`read` in class `InputStream`

Returns:

the next byte of data, or -1 if the end of the file is reached.

Throws:

`IOException` - if an I/O error occurs.

```
package FileHandling.BinaryFileHandling;
import java.util.*;
import java.io.*;

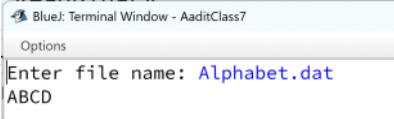
/**
 * Write a description of class ReadBinary here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ReadBinary
{
    public static void main(String arga[]){
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter file name: ");
        String path = "./BinaryFiles/";
        String fileName = sc.nextLine();
        int ch;
        try(FileInputStream fis = new FileInputStream(path+fileName)){
```

```

        while((ch = fis.read()) != -1){
            System.out.print((char)ch);
        }
    }
    catch(IOException ioe){
        System.out.print("\nNetwork error!");
    }
}
}

```

Output:



Writing Primitive Data to a Binary File

Suppose we want to store:

- Roll Number (int)
- Name(string)
- Marks(float)

Class DataOutputStream

```

java.lang.Object
  java.io.OutputStream
    java.io.FilterOutputStream
      java.io.DataOutputStream

```

All Implemented Interfaces:

Closeable, DataOutput, Flushable, AutoCloseable

```

public class DataOutputStream
  extends FilterOutputStream
  implements DataOutput

```

A data output stream lets an application write primitive Java data types to an output stream in a portable way. An application can then use a data input stream to read the data back in. A data output stream wraps another output stream and delegates writing bytes to the write methods of that output stream. Writing data consisting of more than a single byte may cause several writes to the underlying output stream.

A DataOutputStream is not safe for use by multiple concurrent threads. If a DataOutputStream is to be used by more than one thread then access to the data output stream should be controlled by appropriate synchronization.

Since:

1.0

Constructors

Constructor	Description
<code>DataOutputStream(OutputStream out)</code>	Creates a new data output stream to write data to the specified underlying output stream.

Method Summary

All Methods Instance Methods Concrete Methods

Modifier and Type	Method	Description
void	<code>flush()</code>	Flushes this data output stream.
final int	<code>size()</code>	Returns the current value of the counter written, the number of bytes written to this data output stream so far.
void	<code>write(byte[] b, int off, int len)</code>	Writes len bytes from the specified byte array starting at offset off to the underlying output stream.
void	<code>write(int b)</code>	Writes the specified byte (the low eight bits of the argument b) to the underlying output stream.
final void	<code>writeBoolean(boolean v)</code>	Writes a boolean to the underlying output stream as a 1-byte value.
final void	<code>writeByte(int v)</code>	Writes out a byte to the underlying output stream as a 1-byte value.
final void	<code>writeBytes(String s)</code>	Writes out the string to the underlying output stream as a sequence of bytes.

final void	<code>writeChar(int v)</code>	Writes a char to the underlying output stream as a 2-byte value, high byte first.
final void	<code>writeChars(String s)</code>	Writes a string to the underlying output stream as a sequence of characters.
final void	<code>writeDouble(double v)</code>	Converts the double argument to a long using the <code>doubleToLongBits</code> method in class <code>Double</code> , and then writes that long value to the underlying output stream as an 8-byte quantity, high byte first.
final void	<code>writeFloat(float v)</code>	Converts the float argument to an int using the <code>floatToIntBits</code> method in class <code>Float</code> , and then writes that int value to the underlying output stream as a 4-byte quantity, high byte first.
final void	<code>writeInt(int v)</code>	Writes an int to the underlying output stream as four bytes, high byte first.
final void	<code>writeLong(long v)</code>	Writes a long to the underlying output stream as eight bytes, high byte first.
final void	<code>writeShort(int v)</code>	Writes a short to the underlying output stream as two bytes, high byte first.
final void	<code>writeUTF(String str)</code>	Writes a string to the underlying output stream using modified UTF-8 encoding in a machine-independent manner.

Reading Primitive Data

Class `DataInputStream`

```
java.lang.Object
  java.io.InputStream
    java.io.FilterInputStream
      java.io.DataInputStream
```

All Implemented Interfaces:

`Closeable`, `DataInput`, `AutoCloseable`

```
public class DataInputStream
extends FilterInputStream
implements DataInput
```

A data input stream lets an application read primitive Java data types from an underlying input stream in a machine-independent way. An application uses a data output stream to write data that can later be read by a data input stream.

A `DataInputStream` is not safe for use by multiple concurrent threads. If a `DataInputStream` is to be used by more than one thread then access to the data input stream should be controlled by appropriate synchronization.

Since:

1.0

Constructor Summary

Constructors

Constructor	Description
<code>DataInputStream(InputStream in)</code>	Creates a <code>DataInputStream</code> that uses the specified underlying <code>InputStream</code> .

Method Summary

All Methods	Static Methods	Instance Methods	Concrete Methods	Deprecated Methods
Modifier and Type	Method	Description		
final int	<code>read(byte[] b)</code>	Reads some number of bytes from the contained input stream and stores them into the buffer array <code>b</code> .		
final int	<code>read(byte[] b, int off, int len)</code>	Reads up to <code>len</code> bytes of data from the contained input stream into an array of bytes.		
final boolean	<code>readBoolean()</code>	See the general contract of the <code>readBoolean</code> method of <code>DataInput</code> .		
final byte	<code>readByte()</code>	See the general contract of the <code>readByte</code> method of <code>DataInput</code> .		
final char	<code>readChar()</code>	See the general contract of the <code>readChar</code> method of <code>DataInput</code> .		
final double	<code>readDouble()</code>	See the general contract of the <code>readDouble</code> method of <code>DataInput</code> .		
final float	<code>readFloat()</code>	See the general contract of the <code>readFloat</code> method of <code>DataInput</code> .		
final void	<code>readFully(byte[] b)</code>	See the general contract of the <code>readFully</code> method of <code>DataInput</code> .		

final void	<code>readFully(byte[] b, int off, int len)</code>	See the general contract of the <code>readFully</code> method of <code>DataInput</code> .
final int	<code>readInt()</code>	See the general contract of the <code>readInt</code> method of <code>DataInput</code> .
final String	<code>readLine()</code>	Deprecated. This method does not properly convert bytes to characters.
final long	<code>readLong()</code>	See the general contract of the <code>readLong</code> method of <code>DataInput</code> .
final short	<code>readShort()</code>	See the general contract of the <code>readShort</code> method of <code>DataInput</code> .
final int	<code>readUnsignedByte()</code>	See the general contract of the <code>readUnsignedByte</code> method of <code>DataInput</code> .
final int	<code>readUnsignedShort()</code>	See the general contract of the <code>readUnsignedShort</code> method of <code>DataInput</code> .
final String	<code>readUTF()</code>	See the general contract of the <code>readUTF</code> method of <code>DataInput</code> .
static final String	<code>readUTF(DataInput in)</code>	Reads from the stream in a representation of a Unicode character string encoded in modified UTF-8 format; this string of characters is then returned as a <code>String</code> .
final int	<code>skipBytes(int n)</code>	See the general contract of the <code>skipBytes</code> method of <code>DataInput</code> .

```
package FileHandling.BinaryFileHandling;
import java.io.*;
import java.util.*;

/**
 * Write a description of class ReadPrimitiveData here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ReadPrimitiveData
{
    public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        System.out.print("\nEnter file name: ");
        String path = "./BinaryFiles/";
        String fileName = sc.nextLine();
        String pathAndFile = path + fileName;
        try(FileInputStream fis = new FileInputStream(pathAndFile);
            DataInputStream dis = new DataInputStream(fis)){
            boolean more = false;
            int rn;
            String name;
            float marks;

            try{
                while(true){
                    rn = dis.readInt();
                    name = dis.readUTF();
                    marks = dis.readFloat();
                    System.out.print("\n"+rn+", "+name+", "+marks);
                }
            }
            catch EOFException eofe{
                System.out.print("\n=====");
                System.out.print("\nEnd of file.");
            }
        }
        catch(IOException e){
            e.printStackTrace();
        }
    }
}
```

Output:

Enter file name: **ClassX.dat**

```

1, Gagan, 76.8
2, Yash, 81.92
3, Harsh, 89.4
4, Uday, 84.85
5, Pankaj, 67.82
=====
End of file.

```

Important: Read data in the same order in which it was written. Otherwise, the program may throw exceptions or produce incorrect values.

Homework do Any 2 Question based on text file using binary files.

Storing Java Objects in Binary Files

Java can store complete objects using **serialization**.

Example Object:

```

Student
  |
  v
Roll
name
Marks
  |
  v
Binary file

```

Serialization: Serialization is the process of converting an object into a sequence of bytes so it can be stored in a binary file or transmitted over a network.

```

Student object
  |
  v
Serialization
  |
  v
Binary files

```

Deserialization: Deserialization is the reverse process of serialization.

```

Binary Files
  |
  v
Deserialization
  |
  v
Student object

```

Serializable Interface

To serialize an object. Its class must implement **Serializable** interface.

Official documentation:

```
public interface Serializable
```

Serializability of a class is enabled by the class implementing the `java.io.Serializable` interface.

Warning: Deserialization of untrusted data is inherently dangerous and should be avoided. Untrusted data should be carefully validated according to the "Serialization and Deserialization" section of the [Secure Coding Guidelines for Java](#)

[SE. Serialization Filtering](#) describes best practices for defensive use of serial filters.

Classes that do not implement this interface will not have any of their state serialized or deserialized. All subtypes of a serializable class are themselves serializable. The serialization interface has no methods or fields and serves only to identify the semantics of being serializable.

It is possible for subtypes of non-serializable classes to be serialized and deserialized. During serialization, no data will be written for the fields of non-serializable superclasses. During deserialization, the fields of non-serializable superclasses will be initialized using the no-arg constructor of the first (bottommost) non-serializable superclass. This constructor must be accessible to the subclass that is being deserialized. It is an error to declare a class `Serializable` if this is not the case; the error will be detected at runtime. A serializable subtype may assume responsibility for saving and restoring the state of a non-serializable supertype's public, protected, and (if accessible) package-access fields. See the [Java Object Serialization Specification](#), section 3.1, for a detailed specification of the deserialization process, including handling of serializable and non-serializable classes.

When traversing a graph, an object may be encountered that does not support the `Serializable` interface. In this case the `NotSerializableException` will be thrown and will identify the class of the non-serializable object.

Classes that require special handling during the serialization and deserialization process must implement special methods with these exact signatures:

```
private void writeObject(java.io.ObjectOutputStream out)
    throws IOException
private void readObject(java.io.ObjectInputStream in)
    throws IOException, ClassNotFoundException;
private void readObjectNoData()
    throws ObjectStreamException;
```

The `writeObject` method is responsible for writing the state of the object for its particular class so that the corresponding `readObject` method can restore it. The default mechanism for saving the Object's fields can be invoked by calling `out.defaultWriteObject`. The method does not need to concern itself with the state belonging to its superclasses or subclasses. State is saved by writing the individual fields to the `ObjectOutputStream` using the `writeObject` method or by using the methods for primitive data types supported by `DataOutput`.

The `readObject` method is responsible for reading from the stream and restoring the classes fields. It may call `in.defaultReadObject` to invoke the default mechanism for restoring the object's non-static and non-transient fields. The `defaultReadObject` method uses information in the stream to assign the fields of the object saved in the stream with the correspondingly named fields in the current object. This handles the case when the class has evolved to add new fields. The method does not need to concern itself with the state belonging to its superclasses or subclasses. State is restored by reading data from the `ObjectInputStream` for the individual fields and making assignments to the appropriate fields of the object. Reading primitive data types is supported by `DataInput`.

The `readObjectNoData` method is responsible for initializing the state of the object for its particular class in the event that the serialization stream does not list the given class as a superclass of the object being deserialized. This may occur in cases where the receiving party uses a different version of the deserialized instance's class than the sending party, and the receiver's version extends classes that are not extended by the sender's version. This may also occur if the serialization stream has been tampered; hence, `readObjectNoData` is useful for initializing deserialized objects properly despite a "hostile" or incomplete source stream.

`Serializable` classes that need to designate an alternative object to be used when writing an object to the stream should implement this special method with the exact signature:

```
ANY-ACCESS-MODIFIER Object writeReplace() throws ObjectStreamException;
```

This `writeReplace` method is invoked by serialization if the method exists and it would be accessible from a method defined within the class of the object being

serialized. Thus, the method can have private, protected and package-private access. Subclass access to this method follows java accessibility rules. Classes that need to designate a replacement when an instance of it is read from the stream should implement this special method with the exact signature.

ANY-ACCESS-MODIFIER Object readResolve() throws ObjectStreamException;

This readResolve method follows the same invocation rules and accessibility rules as writeReplace.

Enum types are all serializable and receive treatment defined by the [Java Object Serialization Specification](#) during serialization and deserialization. Any declarations of the special handling methods discussed above are ignored for enum types.

Record classes can implement Serializable and receive treatment defined by the [Java Object Serialization Specification, Section 1.13, "Serialization of Records"](#). Any declarations of the special handling methods discussed above are ignored for record types.

The serialization runtime associates with each serializable class a version number, called a serialVersionUID, which is used during deserialization to verify that the sender and receiver of a serialized object have loaded classes for that object that are compatible with respect to serialization. If the receiver has loaded a class for the object that has a different serialVersionUID than that of the corresponding sender's class, then deserialization will result in an [InvalidClassException](#). A serializable class can declare its own serialVersionUID explicitly by declaring a field named "serialVersionUID" that must be static, final, and of type long:

ANY-ACCESS-MODIFIER **static final long serialVersionUID = 42L;**

If a serializable class does not explicitly declare a serialVersionUID, then the serialization runtime will calculate a default serialVersionUID value for that class based on various aspects of the class, as described in the [Java Object Serialization Specification](#). This specification defines the serialVersionUID of an enum type to be 0L. However, it is *strongly recommended* that all serializable classes other than enum types explicitly declare serialVersionUID values, since the default serialVersionUID computation is highly sensitive to class details that may vary depending on compiler implementations, and can thus result in unexpected InvalidClassExceptions during deserialization. Therefore, to guarantee a consistent serialVersionUID value across different java compiler implementations, a serializable class must declare an explicit serialVersionUID value. It is also strongly advised that explicit serialVersionUID declarations use the private modifier where possible, since such declarations apply only to the immediately declaring class--serialVersionUID fields are not useful as inherited members. Array classes cannot declare an explicit serialVersionUID, so they always have the default computed value, but the requirement for matching serialVersionUID values is waived for array classes.

Since:

1.1

From <<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/Serializable.html>>

```
package FileHandling.BinaryFileHandling;
import java.io.*;

/**
 * Write a description of class Student here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Student implements Serializable
{
    private static final long serialVersionUID = 1L;

    private int roll;
    private String name;
```

```

private double marks;

public Student(int roll, String name, double marks){
    this.roll = roll;
    this.name = name;
    this.marks = marks;
}

@Override
public String toString(){
    return "\nRoll: " + roll + "\nName: " + name +
        "\nMarks: "+marks+"\n";
}
}

```

Serializable is a marker interface. It has no methods but tells the JVM that objects of this class may be serialized.

Writing an Object to a Binary File

ObjectOutputStream class:

public class `ObjectOutputStream` extends [OutputStream](#) implements [ObjectOutput](#), [ObjectStreamConstants](#)

An `ObjectOutputStream` writes primitive data types and graphs of Java objects to an `OutputStream`. The objects can be read (reconstituted) using an `ObjectInputStream`. Persistent storage of objects can be accomplished by using a file for the stream. If the stream is a network socket stream, the objects can be reconstituted on another host or in another process.

Only objects that support the `java.io.Serializable` interface can be written to streams. The class of each serializable object is encoded including the class name and signature of the class, the values of the object's fields and arrays, and the closure of any other objects referenced from the initial objects.

The method `writeObject` is used to write an object to the stream. Any object, including Strings and arrays, is written with `writeObject`. Multiple objects or primitives can be written to the stream. The objects must be read back from the corresponding `ObjectInputStream` with the same types and in the same order as they were written.

Primitive data types can also be written to the stream using the appropriate methods from `DataOutput`. Strings can also be written using the `writeUTF` method.

The default serialization mechanism for an object writes the class of the object, the class signature, and the values of all non-transient and non-static fields. References to other objects (except in transient or static fields) cause those objects to be written also. Multiple references to a single object are encoded using a reference sharing mechanism so that graphs of objects can be restored to the same shape as when the original was written.

For example to write an object that can be read by the example in [ObjectInputStream](#):

```

try (FileOutputStream fos = new FileOutputStream("t.tmp");
    ObjectOutputStream oos = new ObjectOutputStream(fos)) {
    oos.writeObject("Today");
    oos.writeObject(LocalDate.now());
} catch (Exception ex) {
    // handle exception
}

```

Serializable classes that require special handling during the serialization and deserialization process should implement methods with the following signatures:

```

private void readObject(java.io.ObjectInputStream stream)
    throws IOException, ClassNotFoundException;
private void writeObject(java.io.ObjectOutputStream stream)

```

```
        throws IOException;
    private void readObjectNoData()
        throws ObjectStreamException;
```

The method name, modifiers, return type, and number and type of parameters must match exactly for the method to be used by serialization or deserialization. The methods should only be declared to throw checked exceptions consistent with these signatures.

The `writeObject` method is responsible for writing the state of the object for its particular class so that the corresponding `readObject` method can restore it. The method does not need to concern itself with the state belonging to the object's superclasses or subclasses. State is saved by writing the individual fields to the `ObjectOutputStream` using the `writeObject` method or by using the methods for primitive data types supported by `DataOutput`.

Serialization does not write out the fields of any object that does not implement the `java.io.Serializable` interface. Subclasses of `Objects` that are not serializable can be serializable. In this case the non-serializable class must have a no-arg constructor to allow its fields to be initialized. In this case it is the responsibility of the subclass to save and restore the state of the non-serializable class. It is frequently the case that the fields of that class are accessible (public, package, or protected) or that there are get and set methods that can be used to restore the state.

Serialization of an object can be prevented by implementing `writeObject` and `readObject` methods that throw the `NotSerializableException`. The exception will be caught by the `ObjectOutputStream` and abort the serialization process.

Implementing the `Externalizable` interface allows the object to assume complete control over the contents and format of the object's serialized form. The methods of the `Externalizable` interface, `writeExternal` and `readExternal`, are called to save and restore the objects state. When implemented by a class they can write and read their own state using all of the methods of `ObjectOutput` and `ObjectInput`. It is the responsibility of the objects to handle any versioning that occurs.

Enum constants are serialized differently than ordinary serializable or externalizable objects. The serialized form of an enum constant consists solely of its name; field values of the constant are not transmitted. To serialize an enum constant, `ObjectOutputStream` writes the string returned by the constant's `name` method. Like other serializable or externalizable objects, enum constants can function as the targets of back references appearing subsequently in the serialization stream. The process by which enum constants are serialized cannot be customized; any class-specific `writeObject` and `writeReplace` methods defined by enum types are ignored during serialization. Similarly, any `serialPersistentFields` or `serialVersionUID` field declarations are also ignored--all enum types have a fixed `serialVersionUID` of 0L.

Primitive data, excluding serializable fields and externalizable data, is written to the `ObjectOutputStream` in block-data records. A block data record is composed of a header and data. The block data header consists of a marker and the number of bytes to follow the header. Consecutive primitive data writes are merged into one block-data record. The blocking factor used for a block-data record will be 1024 bytes. Each block-data record will be filled up to 1024 bytes, or be written whenever there is a termination of block-data mode. Calls to the `ObjectOutputStream` methods `writeObject`, `defaultWriteObject` and `writeFields` initially terminate any existing block-data record.

Records are serialized differently than ordinary serializable or externalizable objects, see [record serialization](#).

Since:
1.1

From <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/ObjectOutputStream.html>

For method information click the above link to read more about it.

ObjectInputStream class:

```
public class ObjectInputStream extends InputStream implements ObjectInput,
```

ObjectStreamConstants

An `ObjectInputStream` deserializes primitive data and objects previously written using an `ObjectOutputStream`.

Warning: Deserialization of untrusted data is inherently dangerous and should be avoided. Untrusted data should be carefully validated according to the "Serialization and Deserialization" section of the [Secure Coding Guidelines for Java SE](#). [Serialization Filtering](#) describes best practices for defensive use of serial filters.

The key to disabling deserialization attacks is to prevent instances of arbitrary classes from being deserialized, thereby preventing the direct or indirect execution of their methods. [ObjectInputFilter](#) describes how to use filters and [ObjectInputFilter.Config](#) describes how to configure the filter and filter factory. Each stream has an optional deserialization filter to check the classes and resource limits during deserialization. The JVM-wide filter factory ensures that a filter can be set on every `ObjectInputStream` and every object read from the stream can be checked. The [ObjectInputStream constructors](#) invoke the filter factory to select the initial filter which may be updated or replaced by `setObjectInputFilter(java.io.ObjectInputFilter)`.

If an `ObjectInputStream` has a filter, the [ObjectInputFilter](#) can check that the classes, array lengths, number of references in the stream, depth, and number of bytes consumed from the input stream are allowed and if not, can terminate deserialization.

`ObjectOutputStream` and `ObjectInputStream` can provide an application with persistent storage for graphs of objects when used with a `FileOutputStream` and `FileInputStream` respectively. `ObjectInputStream` is used to recover those objects previously serialized. Other uses include passing objects between hosts using a socket stream or for marshaling and unmarshaling arguments and parameters in a remote communication system.

`ObjectInputStream` ensures that the types of all objects in the graph created from the stream match the classes present in the Java Virtual Machine. Classes are loaded as required using the standard mechanisms.

Only objects that support the `java.io.Serializable` or `java.io.Externalizable` interface can be read from streams.

The method `readObject` is used to read an object from the stream. Java's safe casting should be used to get the desired type. In Java, strings and arrays are objects and are treated as objects during serialization. When read they need to be cast to the expected type.

Primitive data types can be read from the stream using the appropriate method on `DataInput`.

The default deserialization mechanism for objects restores the contents of each field to the value and type it had when it was written. Fields declared as `transient` or `static` are ignored by the deserialization process. References to other objects cause those objects to be read from the stream as necessary. Graphs of objects are restored correctly using a reference sharing mechanism. New objects are always allocated when deserializing, which prevents existing objects from being overwritten.

Reading an object is analogous to running the constructors of a new object. Memory is allocated for the object and initialized to zero (NULL). No-arg constructors are invoked for the non-serializable classes and then the fields of the serializable classes are restored from the stream starting with the serializable class closest to `java.lang.Object` and finishing with the object's most specific class.

For example to read from a stream as written by the example in [ObjectOutputStream](#):

```
try (FileInputStream fis = new FileInputStream("t.tmp");
    ObjectInputStream ois = new ObjectInputStream(fis)) {
    String label = (String) ois.readObject();
    LocalDateTime dateTime = (LocalDateTime) ois.readObject();
    // Use label and dateTime
} catch (Exception ex) {
    // handle exception
}
```

Classes control how they are serialized by implementing either the `java.io.Serializable` or `java.io.Externalizable` interfaces.

Implementing the `Serializable` interface allows object serialization to save and restore the entire state of the object and it allows classes to evolve between the time the stream is written and the time it is read. It automatically traverses references between objects, saving and restoring entire graphs.

`Serializable` classes that require special handling during the serialization and deserialization process should implement methods with the following signatures:

```
private void writeObject(java.io.ObjectOutputStream stream)
    throws IOException;
private void readObject(java.io.ObjectInputStream stream)
    throws IOException, ClassNotFoundException;
private void readObjectNoData()
    throws ObjectStreamException;
```

The method name, modifiers, return type, and number and type of parameters must match exactly for the method to be used by serialization or deserialization. The methods should only be declared to throw checked exceptions consistent with these signatures.

The `readObject` method is responsible for reading and restoring the state of the object for its particular class using data written to the stream by the corresponding `writeObject` method. The method does not need to concern itself with the state belonging to its superclasses or subclasses. State is restored by reading data from the `ObjectInputStream` for the individual fields and making assignments to the appropriate fields of the object. Reading primitive data types is supported by `DataInput`.

Any attempt to read object data which exceeds the boundaries of the custom data written by the corresponding `writeObject` method will cause an `OptionalDataException` to be thrown with an `eof` field value of `true`. Non-object reads which exceed the end of the allotted data will reflect the end of data in the same way that they would indicate the end of the stream: bitwise reads will return `-1` as the byte read or number of bytes read, and primitive reads will throw `EOFExceptions`. If there is no corresponding `writeObject` method, then the end of default serialized data marks the end of the allotted data.

Primitive and object read calls issued from within a `readExternal` method behave in the same manner--if the stream is already positioned at the end of data written by the corresponding `writeExternal` method, object reads will throw `OptionalDataExceptions` with `eof` set to `true`, bitwise reads will return `-1`, and primitive reads will throw `EOFExceptions`. Note that this behavior does not hold for streams written with the old `ObjectStreamConstants.PROTOCOL_VERSION_1` protocol, in which the end of data written by `writeExternal` methods is not demarcated, and hence cannot be detected.

The `readObjectNoData` method is responsible for initializing the state of the object for its particular class in the event that the serialization stream does not list the given class as a superclass of the object being deserialized. This may occur in cases where the receiving party uses a different version of the deserialized instance's class than the sending party, and the receiver's version extends classes that are not extended by the sender's version. This may also occur if the serialization stream has been tampered; hence, `readObjectNoData` is useful for initializing deserialized objects properly despite a "hostile" or incomplete source stream.

Serialization does not read or assign values to the fields of any object that does not implement the `java.io.Serializable` interface. Subclasses of `Objects` that are not serializable can be serializable. In this case the non-serializable class must have a no-arg constructor to allow its fields to be initialized. In this case it is the responsibility of the subclass to save and restore the state of the non-serializable class. It is frequently the case that the fields of that class are accessible (public, package, or protected) or that there are get and set methods that can be used to restore the state.

Any exception that occurs while deserializing an object will be caught by the `ObjectInputStream` and abort the reading process.

Implementing the `Externalizable` interface allows the object to assume complete control over the contents and format of the object's serialized form. The methods of the `Externalizable` interface, `writeExternal` and `readExternal`, are called to save and

restore the objects state. When implemented by a class they can write and read their own state using all of the methods of `ObjectOutput` and `ObjectInput`. It is the responsibility of the objects to handle any versioning that occurs.

Enum constants are deserialized differently than ordinary serializable or externalizable objects. The serialized form of an enum constant consists solely of its name; field values of the constant are not transmitted. To deserialize an enum constant, `ObjectInputStream` reads the constant name from the stream; the deserialized constant is then obtained by calling the static method

`Enum.valueOf(Class, String)` with the enum constant's base type and the received constant name as arguments. Like other serializable or externalizable objects, enum constants can function as the targets of back references appearing subsequently in the serialization stream. The process by which enum constants are deserialized cannot be customized: any class-specific `readObject`, `readObjectNoData`, and `readResolve` methods defined by enum types are ignored during deserialization.

Similarly, any `serialPersistentFields` or `serialVersionUID` field declarations are also ignored--all enum types have a fixed `serialVersionUID` of 0L.

Records are serialized differently than ordinary serializable or externalizable objects. During deserialization the record's canonical constructor is invoked to construct the record object. Certain serialization-related methods, such as `readObject` and `writeObject`, are ignored for serializable records. See [Java Object Serialization Specification, Section 1.13, "Serialization of Records"](#) for additional information.

Since:

1.1

From <<https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/io/ObjectInputStream.html>>

For other method click on the above link.

readObject

```
public final Object readObject()
    throws IOException,
    ClassNotFoundException
```

Read an object from the `ObjectInputStream`. The class of the object, the signature of the class, and the values of the non-transient and non-static fields of the class and all of its supertypes are read. Default deserializing for a class can be overridden using the `writeObject` and `readObject` methods. Objects referenced by this object are read transitively so that a complete equivalent graph of objects is reconstructed by `readObject`.

The root object is completely restored when all of its fields and the objects it references are completely restored. At this point the object validation callbacks are executed in order based on their registered priorities. The callbacks are registered by objects (in the `readObject` special methods) as they are individually restored.

The deserialization filter, when not null, is invoked for each object (regular or class) read to reconstruct the root object. See `setObjectInputFilter` for details.

Exceptions are thrown for problems with the `InputStream` and for classes that should not be deserialized. All exceptions are fatal to the `InputStream` and leave it in an indeterminate state; it is up to the caller to ignore or recover the stream state.

Specified by:

`readObject` in interface `ObjectInput`

Returns:

the object read from the stream

Throws:

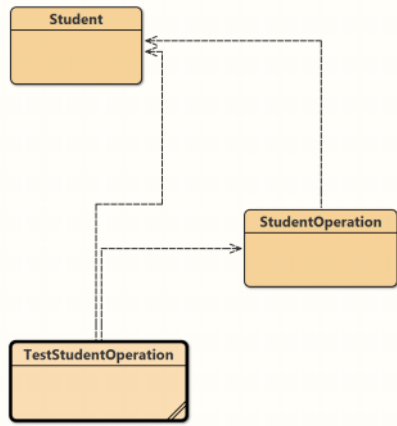
`ClassNotFoundException` - Class of a serialized object cannot be found.

`InvalidClassException` - Something is wrong with a class used by deserialization.

`StreamCorruptedException` - Control information in the stream is inconsistent.

`OptionalDataException` - Primitive data was found in the stream instead of objects.

`IOException` - Any of the usual Input/Output related exceptions.



```

package FileHandling.BinaryFileHandling;
import java.io.*;

/**
 * Write a description of class Student here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Student implements Serializable
{
    private static final long serialVersionUID = 1L;

    private int roll;
    private String name;
    private double marks;

    public Student(int roll, String name, double marks){
        this.roll = roll;
        this.name = name;
        this.marks = marks;
    }

    @Override
    public String toString(){
        return "\nRoll: " + roll + "\nName: " + name +
            "\nMarks: "+marks+"\n";
    }
}

package FileHandling.BinaryFileHandling;
import java.io.*;

/**
 * Write a description of class StudentOperation here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class StudentOperation
{
    private String path = "./BinaryFiles/";

    public void writeObject(String fileName, Student obj){

        try(ObjectOutputStream oos = new ObjectOutputStream(

```

```

        new FileOutputStream(path+fileName))){
            oos.writeObject(obj);
            System.out.print("\nObject saved successfully.");
        }
        catch(IOException ioe){
            System.err.print("\n" + ioe);
        }
    }

    public void readObject(String fileName){
        try(ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream(path+fileName))){
            try
            {
                Student s = (Student)ois.readObject();
                System.out.print("\n" + s);
            }
            catch (ClassNotFoundException cnfe)
            {
                cnfe.printStackTrace();
            }

        }
        catch(FileNotFoundException fnfe){
            System.err.print("\nFile is not found on Specified location.");
        }
        catch(IOException ioe){
            System.err.print("\n" + ioe);
        }
    }
}

```

```

package FileHandling.BinaryFileHandling;
import java.util.*;

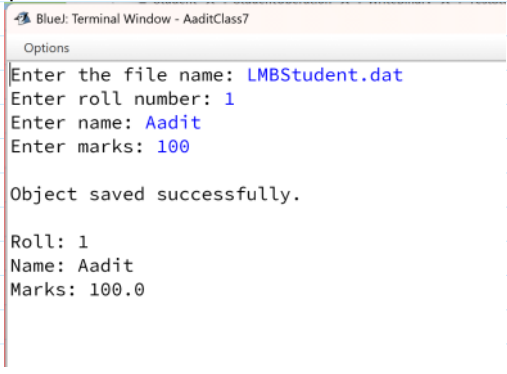
```

```

/**
 * Write a description of class TestStudentOperation here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TestStudentOperation
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String fileName;
        StudentOperation so = new StudentOperation();
        System.out.print("\nEnter the file name: ");
        fileName = sc.nextLine();
        System.out.print("Enter roll number: ");
        int r = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter name: ");
        String nm = sc.nextLine();
        System.out.print("Enter marks: ");
        double m = sc.nextDouble();
        so.writeObject(fileName, new Student(r, nm, m));
        so.readObject(fileName);
    }
}

```

Output:



```
BlueJ: Terminal Window - AaditClass7
Options
Enter the file name: LMBStudent.dat
Enter roll number: 1
Enter name: Aadit
Enter marks: 100

Object saved successfully.

Roll: 1
Name: Aadit
Marks: 100.0
```